

Instituto Tecnológico CTC Colonia

**Ingeniería de
Software**

Obligatorio

Andrés Klett

Paula Camacho

Cristhian Castro

2026

Índice

Índice	2
Declaración de Autoría	4
Abstract	5
Introducción	6
Presentación del cliente	7
Presentación del problema	8
Actores involucrados	9
Lista de necesidades	10
User stories	11
Lista de objetivos	12
Requerimientos funcionales	13
Requerimientos no funcionales	14
Descripción del entorno y análisis de alternativas	15
Metodología de trabajo	16
Un poco de contexto	16
Los cinco principios Lean	16
Enfoque de trabajo	18
Herramientas y prácticas utilizadas	20
Propuesta de diseño	21
Selección de herramientas y tecnologías	23
Entorno de Desarrollo integrado (IDE): Antigravity	23
Framework y arquitectura: ASP.NET Core MVC (.NET 9.0)	23
Mapeador objeto-relacional (ORM): Entity Framework Core (EF Core 9)	23
Motor de Base de Datos: SQL Server (Fase Inicial) / PostgreSQL(Producción)	23
Prototipado y diseño de interfaces (UI/UX): Stitch	23
Plataforma de alojamiento y despliegue: Render	24
Factibilidades	25
Factibilidad técnica:	25
No viable:	25
Análisis de alternativas:	25
Factibilidad económica:	25
Integrantes y roles del proyecto	26
El equipo y el cliente:	26
Dinámica de trabajo: pair programming	26
Descripción de roles y responsabilidades	27
Líder técnico y desarrollador back-end principal	27
Responsable de documentación e ingeniería de requerimientos	27
Arquitecta de base de datos, aseguramiento de calidad y front-end	27
Desarrollo cruzado y de apoyo	27
Estándares y Prácticas del Proyecto	28
Código fuente	28
Base de datos	28

Documentación	29
Interfaces de usuario	29
Pruebas	29
Gestión de configuración	30
Aseguramiento de la Calidad de Software (SQA)	31
Objetivos de Calidad (Metas medibles)	31
Estándares y las 5S en el Código	31
Herramientas y Mapeo Eficiente (MER)	31
Pruebas y Flujo Continuo	31
Gestión de Errores y Mejora (Kaizen)	31
Plan de Pruebas (Plan de Testing)	33
Enfoque:	33
Pruebas de Caja Negra (Funcionales)	33
Pruebas de Caja Blanca (Estructurales)	34
Gestión de Configuración de Software (SCM): Cierre y Documentación	35
Formulario de Cierre Formal	35
Trazabilidad en el Repositorio y Configuración	36
Plan de Capacitación	37
Matriz de Entrenamiento	37
Estrategia de Evaluación y Mejora (Kaizen)	38
Incrementos o iteraciones definidas	39
Cronograma de trabajo y criticidad	42
Ficha cronológica:	42

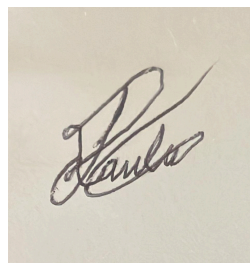
Declaración de Autoría

Nosotros, Cristhian Castro y Paula Camacho, declaramos que el trabajo que se presenta es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida mientras cursamos el tercer semestre de analista programador en la materia Ing. de software con el docente Andrés Klett;
- Hemos tenido en cuenta las clases dictadas por el docente Andrés Klett, quien además proporcionó el material;
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad;
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra;
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente que fue construido por otros, y qué fue contribuido por nosotros;
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Cristhian Castro



Paula Camacho

Abstract

Apicultor, emprendedor de una empresa pequeña que exporta miel, desea automatizar su trabajo y mantener un seguimiento preciso y seguro de sus operaciones. Por ello el objetivo es resolver cuáles son sus problemáticas existentes y crear un software que se ajuste a sus necesidades.

Introducción

Zánganos S.A. es una empresa unipersonal fundada y dirigida por Matías Verges, apicultor dedicado a la producción y exportación de miel al por mayor. Su actividad se desarrolla en un monte de eucalipto en la zona de Rivera, donde gestiona un total de ochocientas colmenas distribuidas en múltiples apiarios, cada uno de ellos conformado por entre cuarenta y sesenta colmenas. Matías es el único trabajador de la empresa y se encarga personalmente de todas las tareas operativas, desde el cuidado y mantenimiento de las colmenas, hasta la cosecha y exportación de la producción.

El trabajo de un apicultor requiere un seguimiento constante y detallado de cada colmena. El proceso de mantenimiento que lleva a cabo Matías, incluye la revisión del estado de la reina y las crías, la evaluación de la fortaleza de la colmena, la detección y tratamiento de enfermedades o plagas, y el reemplazo o incorporación de bastidores. Este proceso se realiza mensualmente, con la excepción del invierno, período en el que el cuidado se vuelve más intensivo y se prioriza la alimentación de las colmenas sin realizar cosechas.

Sin embargo, hasta la fecha Matías no ha utilizado ninguna herramienta que le permita registrar o hacer seguimiento de su trabajo. La ausencia de un sistema de gestión genera problemas concretos y recurrentes en su operativa diaria: no siempre puede determinar de qué colmena proviene cada barril cosechado. Olvida administrar segundas dosis de tratamientos sanitarios. Pierde el rastro de qué colmenas ya revisó durante una visita, y frecuentemente llega a los apiarios sin el equipamiento necesario. A su vez, la falta de registros históricos le impide entender por qué algunas cosechas rinden más que otras o identificar cuáles de sus colmenas son más productivas.

Frente a esta situación, el objetivo del presente proyecto es desarrollar un software a medida que permita a Matías centralizar y digitalizar la gestión de todas sus operaciones. La solución busca brindar trazabilidad sobre su producción, control sanitario de cada colmena, planificación de visitas y un historial que le permita tomar decisiones informadas para mejorar el rendimiento de su empresa a lo largo del tiempo.

Presentación del cliente

Matías Verges, tiene una empresa unipersonal localizada en San José, Zánganos S.A. y los campos donde tiene sus apiarios están en Rivera. Matías, se encarga de la producción y exportación de miel al por mayor. Como apicultor responsable de la producción, realiza los procesos de cuidado y mantención de los apiarios. Están ubicados en un monte de eucalipto en Rivera.

El proceso de mantención de las colmenas consta de cuatro pasos:

1. Revisión de la reina y crías

Lo primero es verificar la presencia y estado de la reina. Si se produjo una enjambrazón, debe colocar una reina nueva y verificar por qué razón sucedió. Luego, al revisar las crías, debe identificar la fertilidad de la reina y la disponibilidad de espacio. Esta revisión ayuda a prevenir las enjambrazones.

2. Observación de la cantidad de abejas

Se evalúa la fortaleza de la colmena y, si están bien, recolecta los bastidores.

3. Revisión de enfermedades

Se inspecciona si existen signos de enfermedad, plagas o problemas sanitarios. En caso de que suceda, se limpia o se aplica una cura.

4. Reemplazo de bastidores

Se reemplazan los bastidores y, si es necesario, se agregan nuevos.

Hasta ahora, el cliente no ha usado ninguna herramienta para guardar su progreso, por lo que suele tener dificultades en el seguimiento del trabajo diario y poca conciencia de sus resultados:

- Cuando realiza los barriles (300 kg c/u), no siempre le queda claro de qué colmena proviene la producción.
- A veces olvida administrar la segunda dosis de tratamientos o medicación.
- Al revisar un apiario, puede olvidarse de qué colmenas ya revisó.
- Frecuentemente olvida llevar bastidores. También suele equivocarse con las herramientas necesarias para cada visita.
- No identifica por qué razón produce más miel en una cosecha que en otra.

Posee ochocientas colmenas que están divididas por apiarios y, en cada uno de ellos, hay de cuarenta a sesenta colmenas; cada una se etiqueta con un número identificador. De cada apiario se registra el nombre, la latitud, la longitud, la cantidad de colmenas y la zona en la que está ubicado.

Las revisiones que realiza son mensuales y, en invierno, lleva a cabo un cuidado intensivo, donde prioriza la alimentación de las colmenas y no realiza cosechas.

Presentación del problema

Actualmente, el apicultor Matías Verges, dueño de la empresa Zánganos SA gestiona la totalidad de sus operaciones de forma manual, sin ningún sistema de registros. Matías, administra ochocientas colmenas distribuidas en múltiples apiarios en la zona de Rivera, siendo el único trabajador de la empresa y responsable de todas las tareas operativas.

La ausencia de herramientas de gestión genera una serie de fallas en su operativa diaria: no siempre puede determinar de qué colmena proviene cada barril cosechado; si aparece algún problema de calidad en el barril: como fermentación, humedad o contaminación, no puede identificar la causa. Sin hacer esta diferenciación no sabe cuáles colmenas rinden más y cuáles menos.

Olvida administrar segundas dosis para los tratamientos de las abejas, esto hace que la enfermedad pueda perdurar y pierda una cantidad significativa de abejas, también la pérdida de la abeja reina o de panales completos, disminuyendo la producción de miel. En consecuencia tendrá mayores costos operativos y deberá invertir en nuevos tratamientos. Cabe destacar, la mala salud de la colmena afecta la calidad de la miel.

Pierde el rastro de qué colmenas ya fueron revisadas durante una visita, y frecuentemente concurre a los apiarios sin el equipamiento o los bastidores necesarios. Este problema hace que el cliente demore más tiempo haciendo su trabajo de campo, repitiendo revisiones y no completando otras. Se expone a los cambios de clima, gastos innecesarios y es menos efectivo a la hora del cuidado y mantención de los apiarios. Haciendo que la producción de miel pueda ser baja o se retrase, afectando así el cumplimiento con sus clientes.

A su vez, no cuenta con registros históricos que le permitan identificar por qué una cosecha rinde más que otra o qué colmenas son históricamente más productivas. Esto hace que el cliente siga invirtiendo tiempo y recursos por igual en todas las colmenas. Tampoco registra cuál es la causa de mejor rendimiento; si rindió más por el clima, por floración o por la salud de las abejas. La falta de registros dificulta la detección de problemas a tiempo y esto es crucial ya que seguir el rendimiento del panal puede indicar enfermedades, sequía, fumigaciones cercanas o problemas con la reina (por ejemplo la adaptación de un enjambre a una reina nueva). Sin este manejo histórico el cliente está expuesto a tomar malas decisiones, como mover colmenas innecesariamente o invertir en equipos que no den resultados. Todo esto hace que a la hora de la venta pierda cantidades significativas de dinero.

El cliente depende de su memoria y sin un sistema que centralice esta información, dichos problemas se seguirán repitiendo visita tras visita, afectando tanto la salud de las colmenas como la capacidad de Zánganos SA para comprender y mejorar su propia producción, imposibilitando su crecimiento económico y patrimonial.

Actores involucrados

Los actores que interactúan con el sistema son:

Matías Verges cuenta con el rol de administrador ya que es el actor principal y único usuario directo del sistema. Como dueño y único trabajador de Zánganos S.A., es quien registra y consulta toda la información relacionada con las colmenas, los apiarios y la producción.

El Ministerio de Ganadería es un actor externo e indirecto. Si bien no utiliza el sistema directamente, es el destinatario de los formularios oficiales que el sistema debe ser capaz de generar para que Matías pueda cumplir con los requisitos administrativos correspondientes.

Lista de necesidades

El primer paso para el diseño del sistema consistió en el relevamiento de las problemáticas y demandas operativas manifestadas por el productor en su actividad diaria. A continuación, se detalla la lista de necesidades identificadas, las cuales abarcan desde el control individualizado de las colmenas hasta la planificación logística de las visitas y el cumplimiento de las normativas legales vigentes.

- Registrar y consultar información individual de cada colmena y de cada apiario.
- Llevar un control del proceso de revisión de cada colmena, sabiendo en todo momento cuáles ya fueron revisadas durante una visita.
- Registrar y hacer seguimiento de los tratamientos sanitarios aplicados por colmena, incluyendo las segundas dosis.
- Identificar el estado de salud de cada colmena, detectando enfermedades, necesidad de cambio de reina o situaciones de enjambrazón.
- Registrar la producción de miel por colmena y por cosecha, con el fin de conocer el origen de cada barril.
- Contar con un historial de producción que permita comparar rendimientos entre cosechas y entre colmenas.
- Planificar las visitas a los apiarios, incluyendo los bastidores y herramientas necesarias a llevar.
- Generar formularios requeridos por el Ministerio de Ganadería.

User stories

Con el propósito de transformar las necesidades detectadas en funcionalidades concretas desde la perspectiva del usuario final, se procedió a la creación de las historias de usuario. Estas representaciones narrativas describen de manera sencilla el rol del actor principal, la acción que desea realizar dentro de la aplicación y el valor de negocio o beneficio que se espera obtener con dicha implementación.

- Como Administrador, quiero registrar la información de cada colmena y apiario, para tener un historial organizado y accesible.
- Como Administrador, quiero saber qué colmenas ya revisé durante una visita, para no perder el rastro ni saltarme ninguna.
- Como Administrador, quiero registrar los tratamientos sanitarios aplicados por colmena, para no olvidar administrar las segundas dosis.
- Como Administrador, quiero poder identificar si una colmena está enferma, necesita cambio de reina o tuvo una enjambrazón, para actuar a tiempo y evitar pérdidas.
- Como Administrador, quiero registrar cuánta miel produjo cada colmena en cada cosecha, para saber el origen de cada barril y comparar rendimientos a lo largo del tiempo.
- Como Administrador, quiero planificar mis visitas indicando los bastidores y herramientas que necesito llevar, para no llegar al apiario sin el equipamiento necesario.
- Como Administrador, quiero generar formularios para el Ministerio de Ganadería, para cumplir con los requisitos administrativos sin tener que hacerlo manualmente.
- Como Administrador, quiero registrar y diferenciar las operaciones según la temporada del año, para adaptar el seguimiento al cuidado intensivo de invierno y a las cosechas del resto del año.

Lista de objetivos

La siguiente lista de objetivos define el impacto esperado que tendrá el software en la optimización de los procesos, la reducción de errores en el campo y la mejora en la toma de decisiones basada en datos históricos.

- Eliminar la pérdida de trazabilidad de producción registrando el origen de cada barril cosechado, logrando un 100% de identificación por colmena.
- Reducir el incumplimiento de tratamientos sanitarios en un 90% mediante notificaciones automáticas de segundas dosis pendientes.
- Reducir considerablemente las visitas al campo sin equipamiento completo incorporando una planificación previa de bastidores y herramientas necesarias para cada visita.
- Permitir el análisis histórico de producción por colmena y por cosecha, facilitando la identificación de las colmenas más productivas y las variaciones entre temporadas.

Requerimientos funcionales

Los requerimientos funcionales definen los servicios, acciones y comportamientos específicos que el sistema web debe ejecutar para dar respuesta a las necesidades de la organización. Esta sección detalla de manera sistemática las capacidades operativas que la aplicación pondrá a disposición del usuario para la gestión de apiarios, controles sanitarios y registros de producción.

- El sistema debe permitir registrar, editar y consultar la información de cada colmena y apiario.
- El sistema debe registrar el proceso de revisión de cada colmena, marcando cuáles fueron revisadas durante una visita y cuáles no.
- El sistema debe registrar los tratamientos sanitarios aplicados por colmena y notificar cuando haya una segunda dosis pendiente.
- El sistema debe permitir registrar el estado de salud de cada colmena, indicando si está enferma, si requiere cambio de reina o si se produjo una enjambrazón.
- El sistema debe registrar la producción de miel por colmena y por cosecha, vinculando cada barril a su colmena de origen.
- El sistema debe mostrar un historial de producción que permita comparar rendimientos entre colmenas y entre cosechas.
- El sistema debe permitir planificar visitas a los apiarios, detallando el equipamiento y bastidores necesarios.
- El sistema debe diferenciar las operaciones según la temporada, adaptándose al modo de cuidado intensivo de invierno.
- El sistema debe permitir generar formularios oficiales para el Ministerio de Ganadería.

Requerimientos no funcionales

A continuación, se especifican los requerimientos no funcionales que garantizan aspectos clave como el rendimiento, la usabilidad para usuarios no digitales y la tolerancia a fallos en entornos rurales.

- El sistema debe responder a las acciones del usuario en menos de 2 segundos.
- El sistema debe ser intuitivo y fácil de usar, dado que el usuario no tiene experiencia con herramientas digitales de gestión.
- El sistema debe respaldar los datos ante pérdidas accidentales.

Descripción del entorno y análisis de alternativas

En una primera instancia el equipo propuso utilizar .NET 8 (por ser la versión más estable y con soporte a largo plazo) como entorno de desarrollo, por ser una plataforma gratuita y de código abierto creada por Microsoft que permite desarrollar, compilar y ejecutar todo tipo de aplicaciones. Como framework de interfaz se evaluó Avalonia UI, un framework comunitario multiplataforma que permite desarrollar aplicaciones de escritorio nativas en Windows, Mac y Linux.

Esta alternativa fue considerada ideal teniendo en cuenta el perfil del usuario final. Matías es una persona con escaso contacto con la tecnología, para quién el uso de wifi, la navegación web o el manejo de dispositivos móviles puede representar una dificultad. Una aplicación de escritorio le permitiría interactuar con el sistema de forma más orgánica e intuitiva, sin depender de una conexión a internet ni de conocimientos técnicos previos.

Para la gestión y persistencia de los datos se seleccionó PostgreSQL como motor de base de datos, alojandose en la nube a través de la plataforma Render.com. Se optó por esta tecnología debido a su robustez, al cumplimiento de los estándares ACID (garantizando la integridad absoluta de los historiales de cosechas y tratamientos) y su capacidad para manejar concurrencia. A diferencia de una base de datos local, esta arquitectura en la nube permite que el sistema opere de forma centralizada

Sin embargo, luego de evaluar la propuesta junto al equipo docente, se identificó que una aplicación de escritorio podría generar inconvenientes al momento de instalarla en distintas máquinas durante la presentación y defensa del proyecto, considerando los tiempos disponibles. Por este motivo el equipo decidió migrar hacia un entorno web, cambiando a .NET 9 como tecnología base porque se trabaja con mapas interactivos. Por lo que se usa SQL Server en un inicio, luego se pasaría a PostgreSQL, como ya se mencionò anteriormente.

Metodología de trabajo

El equipo adopta Scrum Simplificado (metodología ágil) como metodología de trabajo. El proyecto se desarrolla siguiendo un flujo iterativo dividido en etapas. Primero se realiza el relevamiento de requisitos mediante entrevistas al cliente. Luego el equipo define y divide las tareas entre sus integrantes. Durante el desarrollo ambos trabajan en paralelo sobre sus tareas asignadas y se reúnen con frecuencia para revisar avances, resolver bloqueos y sincronizar el trabajo. Al finalizar cada etapa se realiza una presentación parcial al cliente para validar que el desarrollo va en la dirección correcta. Una vez completadas todas las etapas se realiza la entrega final del producto.

Por otra parte, la metodología que se adopta como inclinación práctica y filosófica, es el método Lean y la técnica Kaizen. Cada uno tiene un enfoque interesante de lo que es crear software simple y efectivo. Sin errores y fácilmente de usar. Kaizen, cuestiona ¿qué mejoramos? En cambio Lean propone ¿qué eliminamos?

Pero, ¿Qué es Lean?

Lean es una filosofía de gestión que tiene como objetivo entregar el máximo valor posible al cliente eliminando todo aquello que no lo genera. A esto se lo llama “desperdicio”, y se manifiesta como tiempo perdido, pasos innecesarios, funcionalidades que nadie usa o recursos mal aprovechados. En otras palabras “Menos es más”, optimizando cada parte del proceso para que solo exista lo estrictamente necesario. No es una regla fija, es una mentalidad de trabajo que ayuda al equipo a crecer y aportar valor real al cliente.

Un poco de contexto

¿Cómo nació Lean?

Después de la segunda guerra mundial, Japón quedó devastado. Las fábricas estaban destruidas, no había dinero, no había recursos ni materiales. Toyota casi quiebra. Tenían que producir autos pero sin desperdiciar ningún recurso. Entonces, Eiji Toyoda, se propuso investigar las fábricas estadounidenses y a su competencia directa, Ford.

Frente a la escasez de recursos, la empresa buscó maximizar el valor entregado al cliente eliminando desperdicios y produciendo únicamente lo necesario. Décadas más tarde, estos principios fueron estudiados por el MIT (Massachusetts Institute of Technology) y adaptados al desarrollo de software como una filosofía de trabajo orientada a la eficiencia y la mejora continua.

Los cinco principios Lean

Definir el valor ¿qué quiere realmente el cliente?

El primer principio de Lean establece que antes de hacer cualquier cosa el equipo debe identificar con precisión qué es lo que el cliente considera valioso.

Seguir el flujo de valor:

Identificar cada paso del proceso. Una vez que se sabe qué quiere el cliente, se analiza cada paso del proceso que se necesita para entregárselo y clasifica cada uno de esos pasos en tres categorías. Los que agregan valor directamente, los que no agregan valor pero son necesarios, y los que no agregan valor y se pueden eliminar.

Ejemplificando con la realidad: Matías visita un apiario hasta que el registro queda guardado en el sistema. Si en ese camino hay pasos innecesarios como cargar datos duplicados o confirmar acciones que podrían ser automáticas, esos pasos se descartan.

Crear flujo continuo, que el trabajo no se detenga:

Ejemplo del proyecto: Matías carga una visita al apiario el sistema lo guía paso a paso sin que tenga que saltar entre pantallas, buscar información en otro lado o repetir datos que ya ingresó antes.

Sistema pull, trabajar solo cuando hay demanda real:

En lugar de producir o desarrollar por anticipado, el sistema pull establece que solo se trabaja cuando hay una demanda real. Nada se hace antes de que sea necesario.

Buscar la perfección, mejora continua siempre(Kaizen):

Una vez que se define el valor, se sigue el flujo, se crea continuidad y aplica el sistema pull, el trabajo no termina. Siempre hay algo que mejorar, siempre hay algo más que eliminar, siempre hay una forma de entregar más valor al cliente.

Citas de principios que guiaron el desarrollo:

"No es suficiente hacer tu mejor esfuerzo. Primero debes saber qué hacer, y luego hacer tu mejor esfuerzo"

— W. Edwards Deming (el americano que inspiró a Toyota).

"Justo a tiempo. Si fabricas lo que necesitas, cuando lo necesitas y en la cantidad que lo necesitas, no necesitas stock"

— Kiichiro Toyoda

"Sin datos, eres solo otra persona con una opinión"

— W. Edwards Deming

W. Edwards Deming, el americano que compartió sus ideas a Japón:

Resumiendo el nacimiento de la técnica Kaizen...

Kaizen nació de la combinación de tres cosas. La necesidad japonesa de reconstruirse con pocos recursos después de la guerra, las enseñanzas de Deming sobre mejora continua y calidad, y la cultura japonesa de búsqueda de la perfección. Toyota lo convirtió en práctica diaria y Masaaki Imai lo documentó y lo dio a conocer al mundo en 1986.

Enfoque de trabajo

se decidió abordar uno de sus pilares y usarlo de fundamento esencial para la creación de software. Las cinco S. Cinco palabras japonesas que nacieron en Toyota y actualmente se usan en equipos de software como en oficinas y fábricas.

Seiri:

Significa Clasificar. Se separa lo que se usa de lo que no y se elimina todo lo que sobra. En software es eliminar código muerto, archivos viejos o dependencias que ya no se usan.

Seiton:

Significa Ordenar. Todo lo que sí se usa tiene que tener un lugar fijo y debe estar fácil de encontrar. En software es tener una estructura de carpetas clara y consistente.

Seiso:

Significa Limpiar. Mantener el espacio de trabajo limpio y ordenado de forma constante, no solo cuando está muy sucio. En software sería hacer refactorización continua del código.

Seiketsu:

Significa Estandarizar. Convertir las tres S anteriores en normas fijas que todos siguen siempre. En software sería tener convenciones de código que todo el equipo respeta.

Shitsuke:

Significa Sostener. Mantener los estándares con disciplina a lo largo del tiempo sin que se vayan degradando. Es la S más difícil porque requiere hábito y compromiso constante.

¿Cómo se manejaron Lean y Kaizen en el proyecto?

Los principios de Lean y Kaizen no se utilizaron únicamente como marco teórico, sino que guiaron las decisiones tomadas durante el desarrollo del sistema.

Desde la perspectiva Lean, el equipo implementó únicamente las funcionalidades que aportaban valor al cliente, evitando desarrollar características que no formaban parte del alcance actual del proyecto. Si bien el modelo de datos se diseñó pensando en la escalabilidad futura, no se incorporaron funcionalidades innecesarias, manteniendo el software simple y fácil de mantener.

La mejora continua propuesta por Kaizen se aplicó mediante reuniones periódicas entre los integrantes del equipo y presentaciones parciales al cliente. En cada iteración se revisaron los avances, se detectaron oportunidades de mejora y se realizaron ajustes antes de continuar con la siguiente etapa del desarrollo.

Los principios Lean también influyeron en el diseño de la base de datos. Por ejemplo, se incorporaron registros históricos para evitar la pérdida de información sobre los movimientos de las colmenas, el cambio de reinas y los tratamientos sanitarios. De esta forma, el sistema genera información útil para la toma de decisiones sin duplicar datos ni realizar procesos innecesarios.

Asimismo, se diseñó una arquitectura preparada para el crecimiento de la empresa. La incorporación del atributo **Cargo** en la entidad **Apicultor** permite que, en el futuro, el sistema soporte distintos roles de usuario sin necesidad de modificar el modelo de datos, respetando el principio Lean de construir una base flexible sin desarrollar funcionalidades que aún no son necesarias.

Finalmente, el equipo aplicó prácticas de organización y calidad como Kanban para la gestión de tareas, Pair Programming para reducir errores durante el desarrollo y las 5S para mantener un entorno de trabajo ordenado y un código limpio, facilitando el mantenimiento y la evolución del sistema.

Herramientas y prácticas utilizadas

Kanban board:

Para trabajar de forma organizada se utilizó Kanban board. Es una herramienta visual de gestión de trabajo que permite ver en un solo lugar el estado de todas las tareas de un proyecto. La palabra Kanban en japonés significa "tarjeta visual" o "señal visual". Se usan post it o columnas para organizar el estado de la tarea, cada tarea, es una tarjeta que se va moviendo que va saltando de una en una a medida que avanza.

Pair programming:

Otra cosa fundamental para el flujo de trabajo es la programación entre pares. Consta de un productor y un navegador. El conductor es el que escribe código y el navegador es el que revisa, piensa y guía en tiempo real. El conductor piensa en todo el contexto mientras el navegador se enfoca en el detalle. Los roles se van rotando todo el tiempo. Es una práctica de XP (Extreme Programming), aunque hoy se usa también dentro de equipos Scrum.

Propuesta de diseño

Las decisiones tomadas para el diseño de la arquitectura están enfocadas en dos pilares fundamentales: la escalabilidad del sistema y la capacidad de llevar un historial detallado de las operaciones de Zánganos S.A. Como el software debe acompañar el crecimiento de la empresa, la estructura se diseñó para que pueda adaptarse sin problemas a la futura incorporación de empleados y al aumento del volumen de producción de miel. Asimismo, se prioriza el registro cronológico de cada evento en el campo, transformando los datos diarios en un archivo histórico valioso.

Este enfoque es compatible con la aplicación de las metodologías Lean y Kaizen utilizadas durante el desarrollo. Siguiendo los principios de Lean, se implementaron únicamente las funcionalidades necesarias para cumplir con los objetivos del proyecto, evitando trabajo innecesario. Al mismo tiempo, el modelo de datos se diseñó con una estructura flexible y escalable, permitiendo futuras ampliaciones sin requerir una reestructuración completa. De esta forma, la arquitectura facilita la mejora continua promovida por Kaizen, brindando una base sólida para el crecimiento y la evolución del sistema sin agregar complejidad innecesaria en la versión actual.

la entidad **Apicultor** lleva el atributo *Cargo* porque, aunque hoy Matías hace todo solo, el día de mañana puede contratar empleados; de esta forma, el software permite que solo Matías (como Administrador) pueda manejar la venta de **Barriles**, separando su rol del de los futuros peones de campo. Por eso también su relación de 1 a N.

Por otro lado, para que no se pierda el rastro de los movimientos en el monte de Rivera, se armó una relación de muchos a muchos entre **Apiarios** y **Colmenas**. Esto se hizo así para que funcione como un historial: cuando una colmena se mueve de un apiario a otro por temas de floración o clima, el sistema no borra el pasado, sino que guarda todo el camino que hizo esa colmena. Así, Matías puede mirar hacia atrás y entender por qué rindió más o menos según el lugar donde estuvo instalada y detectar enfermedades o pesticidas. La relación que hay entre estas entidades **Tiene** el atributo de la *Fecha_Instalacion* junto con la ubicación (*Fila*, *Columna* y *Sector*) ayudan a trazar el historial de forma correcta.

El diseño incluye restricciones específicas que aseguran la buena inserción de datos y que sean intuitivos para Matías. En la tabla **Colmenas**, el campo *Fortaleza_Abejas* está limitado únicamente a los valores 'Fuerte', 'Media' o 'Débil', haciendo un estándar para el criterio de evaluación de las abejas.

También, en la tabla **Planifica** el sistema exige ingresar de forma obligatoria la cantidad de herramientas y bastidores para cada visita; esto fuerza una preparación previa y soluciona el problema de llegar al apiario sin el equipamiento necesario.

Siguiendo la misma lógica de control histórico, la relación entre **Colmenas** y **Reinas** también se diseñó como una relación de muchos a muchos N a N a través de la tabla **Asigna**, incorporando el atributo *Fecha_Asignacion*. En la práctica de la apicultura, las

reinas se cambian cuando envejecen, bajan su producción o por una enjambrazón. Al registrar el día y la hora exacta en que una reina entra a una colmena, el sistema no borra el pasado cuando Matías introduce una reina nueva, sino que guarda la línea de tiempo completa. Esto resuelve el problema de la falta de registros históricos, permitiendo a Matías analizar el rendimiento del panal hacia atrás y descubrir qué reinas o qué genéticas dieron los mejores resultados en cada cosecha.

Por último, al requerir la fecha exacta en la tabla **Aplica**, la base de datos registra el momento exacto en que se inicia un tratamiento sanitario, permitiendo controlar con precisión el vencimiento y la aplicación de las segundas dosis para proteger la salud de las colmenas.

Selección de herramientas y tecnologías

Entorno de Desarrollo integrado (IDE): Antigravity

El equipo eligió Antigravity como IDE principal por ser la herramienta agéntica utilizada en el flujo de trabajo diario. Para el soporte de desarrollo y depuración en .NET 9.0 / C#, se integró con la consola de .NET CLI y extensiones compatibles con su entorno de ejecución.

Si bien es un buen copiloto se debe utilizar con conciencia para no generar IA slop (código de mala calidad) o código spaghetti; es el término más común para el código desorganizado y difícil de entender. Estos problemas pueden atentar contra la seguridad del sistema y sus datos.

Framework y arquitectura: ASP.NET Core MVC (.NET 9.0)

Para estructurar el proyecto el equipo optó por ASP.NET Core MVC. El patrón Modelo-Vista-Controlador les resultó muy práctico porque divide claramente las responsabilidades del sistema, separando el manejo de datos, la interfaz visual y la lógica de control. También valoraron que es multiplataforma, maneja bien la alta concurrencia de usuarios y al utilizar la versión .NET 9.0 se aseguraron el soporte oficial de Microsoft a largo plazo.

Mapeador objeto-relacional (ORM): Entity Framework Core (EF Core 9)

En lugar de escribir consultas SQL manualmente, el equipo utilizó Entity Framework Core para manejar la base de datos directamente desde las clases en C#. Los cambios estructurales en los datos los gestionaron mediante migraciones, lo que les proporcionó un historial versionado y seguro para revertir modificaciones en caso de ser necesario. Esta herramienta les permite además cambiar el motor de base de datos sin necesidad de reescribir la lógica de la aplicación.

Motor de Base de Datos: SQL Server (Fase Inicial) / PostgreSQL(Producción)

El equipo planteó una estrategia de datos en dos etapas. Durante el desarrollo utilizaron SQL Server por ser una herramienta que ya conocían junto a sus herramientas asociadas. Sin embargo para el despliegue en producción migraron a PostgreSQL debido a restricciones prácticas del entorno de alojamiento seleccionado.

Prototipado y diseño de interfaces (UI/UX): Stitch

Para definir la interfaz gráfica antes de comenzar el desarrollo, el equipo utilizó la plataforma colaborativa en la nube Stitch. Esto les permitió diseñar pantallas, dashboards y formularios con un alto nivel de detalle, aplicando una paleta de tonos tierra para mantener la coherencia visual con la temática apícola del proyecto. La elaboración de estos prototipos les permitió validar la usabilidad del sistema de forma temprana y les proporcionó una referencia visual clara que guió el desarrollo desde el inicio.

Plataforma de alojamiento y despliegue: Render

Para alojar la aplicación web en producción el equipo decidió utilizar Render, una plataforma en la nube moderna que simplifica el despliegue continuo de aplicaciones y servicios web. El motivo principal de esta elección fue la confiabilidad comprobada, ya que el equipo había utilizado esta plataforma durante el cursado para alojar un trabajo práctico anterior con resultados satisfactorios. Al estar ya familiarizados con su entorno y su integración con repositorios de código, garantizaron un proceso de despliegue fluido y sin contratiempos. Además Render gestiona de forma automática los certificados de seguridad HTTPS y se adapta perfectamente a la decisión de utilizar PostgreSQL, permitiéndoles mantener el proyecto en línea de forma rápida y estable.

Factibilidades

Factibilidad técnica:

Un punto de inflexión fue la decisión precipitada de las tecnologías a utilizar sin haber antes planteado las diferentes dificultades. Se creyó conveniente hacer una aplicación de escritorio para que el usuario trabaje fácilmente sin tener conexión a internet, pero no se tomó en cuenta el límite de tiempo y la falta de determinados conocimientos.

No viable:

En primer lugar, se empezó a trabajar con Avalonia, que es la interfaz de usuario y SQLite para poder interactuar con la base de datos de forma local. Sin embargo, ninguno de los integrantes tiene conocimientos sobre el framework Avalonia UI, por lo que crear un software desde cero significa aprenderlo y carecemos de tiempo.

En segundo lugar, la instalación de los proyectos de escritorio pueden ser problemáticos porque la app necesita ciertos componentes instalados en el sistema para funcionar; .Net, librerías o drivers. Esto dificulta el intercambio de datos entre diferentes computadoras. Los permisos de administrador o los antivirus pueden detener la presentación.

Análisis de alternativas:

Cómo es obligatoria la presentación del proyecto y el tiempo es un limitante, el equipo identificó crear una página web con .Net 9, usando el modelo MVC(Model View Controller) y alojarla en Render. Esto facilita la conexión y el manejo de la base de datos(PostgreSQL).

La factibilidad es viable con condiciones, porque en teoría una aplicación de escritorio encaja perfectamente con el perfil del usuario. Es un hombre de unos cincuenta años, con pocos conocimientos de tecnologías, que maneja una empresa unipersonal. Las condiciones son coherentes para instalar el software en una computadora sin internet y si desea migrar, algo que no es su idea por el momento, puede acudir a ayuda técnica, pero la dificultad es que el equipo no cuenta con los conocimientos suficientes para lograr tal ideal.

Factibilidad económica:

Desde la perspectiva del equipo de desarrollo el proyecto no representa ningún costo económico. Todas las herramientas y tecnologías seleccionadas son gratuitas. Antigravity es un IDE de uso libre, .NET 9 es de código abierto y sin costo de licencia, PostgreSQL no requiere ningún tipo de licencia paga, Render ofrece un plan gratuito suficiente para alojar el proyecto y Stitch fue utilizado en su versión sin costo.

Integrantes y roles del proyecto

En esta sección se presenta al equipo detrás del proyecto y cómo se distribuyen las responsabilidades. Es importante destacar que el equipo trabaja de forma altamente colaborativa, ambos integrantes participan, se apoyan y desarrollan tareas en todas las áreas del proyecto. Sin embargo para mantener el orden, la eficiencia y asegurar la calidad, definieron áreas de enfoque principal y responsabilidades finales de la siguiente manera.

El equipo y el cliente:

El proyecto consiste en un sistema web de gestión apícola desarrollado para el cliente Matías Verges, quien será el usuario final. El equipo de desarrollo está conformado por dos integrantes.

Cristhian Castro asume el rol de líder técnico, con enfoque principal en el back-end, revisión de código, gestión de versiones y aseguramiento de calidad.

Paula Camacho es la responsable principal de la documentación, revisión de código, arquitectura de la base de datos, aseguramiento de calidad y líder del front-end.

Dinámica de trabajo: pair programming

Como práctica de desarrollo el equipo aplica la técnica de pair programming, que consiste en que ambos integrantes trabajan sobre la misma tarea al mismo tiempo compartiendo pantalla. Esta técnica define dos roles que se alternan de forma rotativa. El conductor es quien escribe el código activamente, mientras que el navegador observa, analiza y guía en tiempo real, pensando en el panorama general y detectando posibles errores antes de que ocurran. Esta dinámica permite reducir errores, transferir conocimiento entre los integrantes y mejorar la calidad del código de forma continua.

Descripción de roles y responsabilidades

Líder técnico y desarrollador back-end principal

El enfoque principal de Cristhian es definir la arquitectura y la lógica del negocio en el servidor utilizando ASP.NET Core MVC. Es la referencia técnica principal del equipo, diseña y programa los controladores y los modelos. Además actúa como filtro de calidad del back-end, revisando y aprobando cualquier código antes de integrarlo al repositorio principal. Administra el control de versiones en GitHub y, en línea con la naturaleza colaborativa del equipo, también participa activamente en la redacción de la documentación del proyecto.

Responsable de documentación e ingeniería de requerimientos

Aunque ambos integrantes redactan y generan contenido para el proyecto, Paula lidera, revisa y aprueba toda la documentación. Tiene la decisión final sobre la redacción de los requerimientos, casos de uso, historias de usuario, glosarios y manuales. Su misión es pulir los textos de ambos integrantes y asegurar que el proyecto cumpla con todos los estándares académicos, de coherencia y de formato que exige el Instituto.

Arquitecta de base de datos, aseguramiento de calidad y front-end

Paula asume el rol de arquitecta de base de datos del proyecto. En este rol es responsable de diseñar la estructura de datos del sistema, definiendo las entidades, sus atributos y las relaciones entre ellas, garantizando que el modelo sea eficiente, coherente y escalable para las necesidades del cliente.

Además su enfoque en el desarrollo está orientado a que el sistema no solo funcione correctamente sino que sea fácil de usar. Valida que el diseño, desde el prototipado en Stitch hasta el código en Razor, HTML y CSS, sea ergonómico e intuitivo. Realiza las pruebas funcionales de la interfaz para comprobar que el usuario pueda interactuar con el sistema sin dificultades, otorgando el visto bueno final a la experiencia visual.

Desarrollo cruzado y de apoyo

Dado que el equipo trabaja de forma conjunta en todas las áreas del proyecto, Paula colabora programando lógicas en C# para el back-end bajo la supervisión y aprobación de Cristhian. De la misma manera Cristhian apoya en la toma de decisiones y ejecución del front-end, bajo el visto bueno de Paula.

Estándares y prácticas del proyecto

Código fuente

Para mantener un desarrollo ordenado y facilitar el mantenimiento del sistema, el equipo adoptó las convenciones oficiales de programación de Microsoft para C#. Estas reglas permiten que todo el código mantenga una estructura uniforme, haciendo que cualquier integrante del equipo pueda comprenderlo y modificarlo con mayor facilidad.

El proyecto fue desarrollado con el patrón de diseño Model-View-Controller (MVC), una de las más utilizadas en ASP.NET Core. Esta arquitectura separa la lógica de negocio, la interfaz de usuario y el manejo de las solicitudes, permitiendo que cada parte del sistema tenga una responsabilidad definida y favoreciendo la escalabilidad y el mantenimiento del software.

Como parte del proceso de validación se utilizaron Data Annotations sobre los modelos y ViewModels. A través de atributos como *[Required]*, *[Key]* y otras anotaciones, el sistema verifica automáticamente que la información ingresada por el usuario cumpla con las reglas establecidas antes de guardarse en la base de datos.

Además, durante todo el desarrollo se respetaron las nomenclaturas recomendadas por Microsoft, utilizando PascalCase para clases, métodos y propiedades, y camelCase para variables locales y parámetros. Esto permite mantener una estructura consistente y facilita la lectura del código.

Base de datos

Antes de comenzar la implementación de la base de datos, el equipo realizó el diseño del Modelo Entidad-Relación (MER), identificando las entidades del dominio, sus atributos y las relaciones entre ellas. Esta etapa fue decisiva para poder coordinar lo que el cliente desea con el sistema. El equipo se apoyó en la filosofía Lean y Kaizen como también tuvo la flexibilidad de trabajar con un modelo escalable.

Una vez validado el modelo, la base de datos se implementó utilizando Entity Framework Core 9, aprovechando su integración con ASP.NET Core para gestionar el acceso a los datos y administrar la evolución del esquema mediante migraciones.

Durante la etapa de desarrollo se utilizó SQL Server, debido a su facilidad de configuración y portabilidad. Sin embargo, el sistema fue diseñado considerando una futura migración a PostgreSQL, un motor de base de datos más robusto y preparado para soportar un mayor volumen de información en un entorno de producción, ya que se necesita guardar permanentemente el registro histórico..

Las relaciones entre las entidades se configuraron utilizando Fluent API dentro de *ApplicationDbContext*, permitiendo establecer restricciones de integridad referencial y definir comportamientos específicos, como impedir la eliminación automática de registros relacionados mediante configuraciones como *OnDelete(DeleteBehavior.Restrict)*.

Asimismo, cada entidad incorpora propiedades de navegación que representan las relaciones existentes entre los distintos objetos del dominio. Esto facilita la realización de consultas, el manejo de las relaciones entre entidades y el mantenimiento del modelo de datos a medida que el sistema evoluciona.

Documentación

Toda la documentación del proyecto se encuentra centralizada dentro de la carpeta Documentación, con el objetivo de mantener organizada toda la información relacionada con el desarrollo del sistema.

Para la documentación técnica se utiliza el formato Markdown (.md), ya que permite escribir documentación de forma sencilla y mantenerla versionada junto con el código fuente. Por otra parte, las entregas académicas y la documentación formal se presentan en formato PDF.

Al encontrarse integrada dentro del repositorio Git, toda la documentación evoluciona junto con el proyecto, permitiendo conocer el historial de modificaciones y mantener la trazabilidad de los cambios realizados durante el desarrollo.

Interfaces de usuario

La interfaz del sistema fue desarrollada utilizando ASP.NET Core Razor Views (.cshtml) junto con HTML5, aprovechando las herramientas propias del framework para integrar la lógica del servidor con la presentación de la información.

El diseño visual se implementó mediante un archivo CSS modular ([site.css](#)), tomando como referencia los prototipos elaborados previamente con la herramienta Stitch. Esto permitió mantener una apariencia consistente entre las diferentes pantallas del sistema.

Como apoyo al diseño de la interfaz se incorporó la tipografía Google Fonts (Inter) y la biblioteca de iconos Font Awesome 6, buscando ofrecer una experiencia visual uniforme y agradable para el usuario.

Durante el desarrollo también se tuvieron presentes criterios básicos de accesibilidad y usabilidad. Se procuró mantener una navegación intuitiva, una correcta jerarquía visual de la información, botones claramente identificables, formularios fáciles de completar y una distribución consistente de los elementos en todas las pantallas del sistema.

Pruebas

La validación del sistema se realizó mediante pruebas funcionales manuales, verificando el correcto funcionamiento de cada una de las funcionalidades desarrolladas antes de incorporarlas a la versión principal del proyecto.

Entre las pruebas realizadas se incluyeron la navegación entre pantallas, el funcionamiento del Dashboard, las operaciones de alta, baja, modificación y consulta (CRUD) de las tareas, la validación de formularios y la comprobación de los principales flujos de uso del sistema.

Si bien actualmente el proyecto no incorpora pruebas automatizadas mediante herramientas como xUnit o NUnit, la estructura de la solución permite incorporarlas en futuras etapas del desarrollo como parte de un proceso de mejora continua.

Gestión de configuración

El control de versiones del proyecto se realiza mediante Git, herramienta que permite registrar cada modificación realizada sobre el código fuente y mantener un historial completo de la evolución del sistema.

El repositorio se encuentra alojado en GitHub, facilitando el trabajo colaborativo entre los integrantes del equipo y permitiendo sincronizar los cambios de manera segura.

El desarrollo se organiza utilizando una rama principal denominada main, sobre la cual se integran las funcionalidades una vez que han sido revisadas y validadas. Este mecanismo forma parte del proceso de Gestión de Configuración de Software (SCM), permitiendo mantener una versión estable del proyecto, controlar los cambios realizados y garantizar la trazabilidad de cada actualización incorporada al sistema.

Aseguramiento de la calidad de Software (SQA)

Para el equipo, la calidad no es solo meterle un test al código al final del día; es una mentalidad que se cruza directamente con su forma de trabajar. Si su metodología se basa en Scrum Simplificado, Lean y Kaizen para hacer software simple, efectivo y sin errores, el plan de SQA es la herramienta que le asegura que realmente lo esté logrando.

A continuación, se explica cómo se organiza el equipo, qué herramientas usa y qué hace en el día a día para cuidar la calidad del sistema.

Objetivos de Calidad (Metas medibles)

- Operaciones robustas: CRUDs estables en Apiarios, Colmenas, Tareas y Cosechas para que el sistema nunca se caiga al registrar la producción.
- Cero Warnings: Meta estricta de eliminar cualquier advertencia de compilación (como las referencias nulas en *ColmenasController*).
- Migraciones limpias: 100% de éxito al correr las migraciones con EF Core 9.

Estándares y las 5S en el Código

Para evitar código "spaghetti" o "IA slop" al usar los asistentes de inteligencia artificial, el equipo se guía por las 5S:

- Seiri (Clasificar) y Seiso (Limpiar): Con Pair Programming el equipo revisa el código en tiempo real, eliminando código muerto y haciendo refactorización continua.
- Seiton (Ordenar) y Seiketsu (Estandarizar): Arquitectura limpia basada en ASP.NET Core 9 MVC, contexto `<Nullable>enable</Nullable>` activado y validación doble (formularios Razor en frontend y *ModelState.IsValid* en backend).

Herramientas y Mapeo Eficiente (MER)

- Ecosistema: El equipo desarrolla en Google Antigravity junto con .NET CLI (*dotnet build/run*).
- Base de Datos Intocable a Mano: Fiel al enfoque Code-First, las clases en C# son el reflejo exacto del MER. El equipo usa SQL Server en desarrollo y migra de forma transparente a PostgreSQL para el despliegue final en Render.
- Métricas: El equipo controla el conteo de warnings en consola, excepciones en tiempo de ejecución (como caídas de *SQLEXPRESS*) y que la IA respete el diseño original del MER.

Pruebas y Flujo Continuo

- Simulación Real: El equipo prueba a mano los flujos de Matías en el campo de forma fluida (Ingresar apiario, revisar colmena, aplicar dosis sanitaria).
- Restricciones: El equipo verifica que el sistema obligue a ingresar las herramientas y bastidores al planificar visitas, solucionando el problema de llegar al monte de Rivera sin equipamiento.

Gestión de Errores y Mejora (Kaizen)

- Criterio: Si salta un bug, se para, se aísla y se corrige antes de avanzar.

- Severidad: Fallos en *Program.cs* o la base de datos son Críticos; problemas con datos de negocio (como perder la fecha de una reina o la trazabilidad de un barril) son Mayores; detalles visuales de la interfaz (diseñada en Stitch) son Menores.

Plan de pruebas (plan de testing)

Enfoque:

Asegurar el máximo valor eliminando fallas (Lean) mediante la verificación continua de flujos lógicos (Kaizen).

Pruebas de Caja Negra (Funcionales)

Diseñadas para validar la experiencia de Matías en el sistema web sin mirar el código interno.

ID	Caso de Prueba (CP)	Pasos Clave	Resultado Esperado
CP-01	Creación Exitosa de Apiario (Camino Feliz)	Ir a Apiarios, Crear Nuevo, Cargar "Apiario Sur" (Ruta 8, km 45) y Guardar.	Se redirige al Index y el nuevo apiario es visible en la tabla.
CP-02	Campos Obligatorios en Tareas	Ir a Tareas y Crear Nueva. Dejar campos vacíos (Título, Fecha) y Guardar.	No se recarga la página. Muestra mensajes de error en rojo y bloquea el guardado.
CP-03	Edición de Estado de Colmena	Ir a Colmenas, Editar "Colmena #1", Cambiar abejas a 55,000 y estado a "En Cuarentena". Guardar.	Redirige al Index. En "Detalles" se visualizan los cambios persistidos correctamente.
CP-04	Eliminación de Tarea	Ir a Tareas, Buscar "Revisión de rutina", Eliminar y Confirmar.	Redirige al Index y la tarea ya no existe en la tabla.
CP-05	Límites en el Calendario	Crear Tarea con fecha 31/12/2026, Ir a Calendario y Avanzar a Diciembre.	La tarea se muestra anclada al día 31 sin desbordar la interfaz ni dar error de carga.

Pruebas de Caja Blanca (Estructurales)

Diseñadas para testear la lógica interna de los controladores en C# mediante mocks de base de datos (Moq).

CB-01: Cobertura de Ramas en Create (POST) - Tareas Inválidas:

- *Código:* `if (model.TareasColmenas == null || !model.TareasColmenas.Any(...))`
- *Setup:* Instanciar *TareasController* enviando un modelo con tareas nulas o vacías.
- *Resultado:* Entra al bloque if, no llama a `_context.SaveChanges()` y retorna un *ViewResult* para recargar el formulario.

CB-02: Cobertura de Caminos en Index (GET) - Ordenamiento:

- *Código:* `if (ordenarPor == "fecha") { ... } else { ... }`
- *Setup:* Cargar 3 tareas ficticias en el mock. Probar llamando a `Index(ordenarPor: "fecha")` y luego a `Index(ordenarPor: "prioridad")`.
- *Resultado:* Con "fecha" ordena por vencimiento; con "prioridad" (o vacío) ordena por nivel de criticidad. *ViewBag.OrdenActual* guarda el parámetro.

CB-03: Manejo de Nulos en GetColmenasPorApiario (GET):

- *Código:* `if (apiario == null) return Json(new List<object>());`
- *Setup:* Llamar al endpoint JSON buscando un "Apiario Inexistente" en un contexto vacío.
- *Resultado:* El buscador da nulo, entra al if y retorna un *JsonResult* con una lista vacía, evitando un quiebre por *NullReferenceException*.

CB-04: Validación de NotFound() en Detalles y Edición:

- *Código:* `if (t == null) return NotFound();`
- *Setup:* Llamar a *Details(id: 99)* teniendo únicamente los IDs 1 y 2 en el mock.
- *Resultado:* La entidad resulta nula, ejecuta el retorno anticipado y devuelve explícitamente un código HTTP 404 (*NotFoundResult*).

CB-05: Verificación de Mapeo Interno en Edit (POST):

- *Código:* Bloque de asignación dentro de `if (tareaExistente != null)`.
- *Setup:* Enviar un *TareaViewModel* modificado (Prioridad = "Alta") al método Edit para la tarea con ID = 1.
- *Resultado:* El controlador mapea los cambios, actualiza los campos, llama a `_context.SaveChanges()` exactamente 1 vez y redirige al Index.

Gestión de configuración de Software (SCM): cierre y documentación

El objetivo de esta etapa es garantizar que cada modificación realizada en el sistema de Zánganos S.A. sea 100% trazable, auditable y no genere código basura (IA slop). El equipo aplica este flujo formal al finalizar cada Solicitud de Cambio (RFC).

Formulario de Cierre Formal

Cada cambio aprobado e integrado se registra bajo la siguiente estructura de datos:

Campo	Especificación / Datos del Proyecto
Versión / Línea Base	Identificador único de compilación (Ej: v1.2.0 (LB-2026-06)).
Fecha de Implementación	Día exacto del despliegue en el entorno correspondiente.
Responsable	Integrante del equipo desarrollador que aplicó el cambio.
Estado Final	[Exitoso] / [Parcial] / [Fallido]
Comentarios de Verificación	Notas técnicas (Ej: "Cambio verificado en pruebas locales. Listo para producción en Render").
Fecha de Cierre	Fecha en la que se da por concluido el ciclo de la solicitud.
Cerrado por	Responsable de validar el proceso completo (Ej: Andrés Klett / Solicitante).

Trazabilidad en el Repositorio y Configuración

Para que el cambio se considere formalmente cerrado, el equipo asegura el cumplimiento de tres pilares en el control de versiones:

Historial en Git: Cada cambio debe estar respaldado por un commit claro en el repositorio, detallando qué archivos se modificaron y quién lo hizo.

Actualización de Elementos de Configuración (ECs): Si el cambio afecta la estructura de datos, se actualiza inmediatamente el MER, los scripts de migración de EF Core 9 para PostgreSQL, y los manuales de usuario en Stitch. No se admiten parches manuales.

Registro de Estado (Log de SCM): La solicitud pasa de estado "En Desarrollo" o "En Pruebas" a "Cerrada e Integrada" en el panel de control del proyecto, quedando disponible para futuras asesorías o auditorías de control de calidad.

Plan de capacitación

Para asegurar que el sistema web de Zánganos S.A. sea adoptado con éxito y se elimine el desperdicio de datos mal cargados (Lean), el equipo diseñó una estrategia de transferencia de conocimiento estructurada en tres niveles:

Matriz de Entrenamiento

Perfil / Usuario	Modalidad	Temario Clave	Entregable Asociado
Personal de Campo (Matías y operarios)	Taller práctico presencial (1 sesión de 1.5 horas).	Login y navegación móvil. Registro de Apiarios y Colmenas en tiempo real. Control de checklist (bastidores/herramientas) para el viaje a Rivera.	Guía Rápida de Bolsillo: PDF optimizado para celular con capturas del diseño de Stitch.
Administrador del Sistema (Gestión de negocio)	Videoconferencia guiada vía Meet (1 sesión de 1 hora).	Gestión y alta de usuarios. Visualización del Calendario de tareas. Control de alertas sanitarias y reportes de trazabilidad de barriles de 300 kg.	Manual de Administración: Documento técnico con la configuración de alertas.
Soporte Técnico Básico	Auto-capacitación mediante documentación.	Monitoreo del despliegue en Render. Respaldo de PostgreSQL	Manual de Despliegue y SCM: Configuración del entorno de producción.

Estrategia de Evaluación y Mejora (Kaizen)

La capacitación no termina cuando se dicta la charla. Para asegurar la calidad del proceso se aplican dos métricas simples:

- Prueba Autónoma: Al final del taller, cada operario deberá registrar de punta a punta un flujo real (Ingresar apiario, revisar colmena, aplicar dosis sanitaria) sin asistencia.
- Soporte Post-Lanzamiento: Durante las primeras dos semanas en producción (Render), se abrirá un canal directo de WhatsApp para resolver dudas operativas al instante, usando el feedback de los usuarios para corregir posibles textos confusos en la interfaz.

Incrementos o iteraciones definidas

Siguiendo la metodología de Scrum Simplificado orientada a la entrega continua de valor (Lean), el desarrollo del sistema se ha dividido en cuatro iteraciones principales. Al finalizar cada iteración, se entregará a Zánganos S.A. un incremento de software 100% funcional.

Iteración 1: Fundación y Trazabilidad Básica (Gestión de Infraestructura)

Objetivo: Establecer la base del sistema para que Matías pueda digitalizar su infraestructura física y abandonar el uso del papel.

Historias de Usuario abordadas:

- Registrar la información de cada colmena y apiario para tener un historial organizado.
- Identificar si una colmena está enferma, necesita cambio de reina o tuvo enjambrazón.
- Incremento: Módulos de ABM (Alta, Baja, Modificación) de Apiarios y Colmenas. El usuario podrá visualizar en una tabla el estado de salud y la ubicación de sus 800 colmenas en tiempo real.

Iteración 2: Logística de Campo y Control Sanitario (Visitas y Tratamientos)

Objetivo: Solucionar los problemas de olvidos logísticos al viajar a Rivera y prevenir la pérdida de abejas por falta de medicación.

Historias de Usuario abordadas:

- Planificar visitas indicando los bastidores y herramientas necesarias.
- Saber qué colmenas ya fueron revisadas durante una visita.
- Registrar tratamientos sanitarios para no olvidar las segundas dosis.
- Incremento: Módulo de planificación de visitas con checklist preventivo de herramientas. Implementación de un registro de revisión in-situ y un panel de alertas tempranas para dosis médicas pendientes.

Iteración 3: Trazabilidad de Producción

Objetivo: Resolver el problema de no saber de dónde proviene cada barril de miel.

Historias de Usuario abordadas:

- Registrar cuánta miel produjo cada colmena para saber el origen de cada barril.
- Incremento: Módulo de cosechas que vincula directamente los kilogramos de los barriles con el ID de la colmena de origen.

Iteración 4: Burocracia y Cierre del Proyecto (Reportes Oficiales)

Objetivo: Automatizar los requerimientos legales para ahorrar tiempo administrativo.

Historias de Usuario abordadas:

- Generar formularios para el Ministerio de Ganadería de forma automática.
- Incremento: Funcionalidad de exportación de datos (PDF/Excel) que toma la información del historial productivo y la formatea según las exigencias del Ministerio de Ganadería.

Iteración 5: Optimización para Entorno Rural y Estadísticas Avanzadas (PWA)

Objetivo: Asegurar que Matías pueda usar el sistema en el medio del monte sin señal y facilitar la toma de decisiones a largo plazo.

Requerimientos abordados:

- Mostrar un historial de producción que permita comparar rendimientos (Historia de Usuario).
- Estar disponible sin conexión a internet en zonas rurales sin señal (Requerimiento No Funcional).
- Incremento: Implementación de un Dashboard visual con gráficas comparativas por apiario. Además, se añade soporte Offline (tecnología Progressive Web App - PWA) que permite guardar los datos localmente en el celular cuando no hay señal, sincronizándolos automáticamente con el servidor al recuperar la conexión.

Cronograma de trabajo y criticidad

En esta sección se presenta la distribución temporal de las actividades y fases del proyecto. La planificación se divide en iteraciones específicas que permiten realizar un seguimiento preciso de los avances.

Criticidad Alta (Bloqueante / Vital): Comprende aquellas actividades y funcionalidades indispensables para la continuidad del proyecto o la operativa básica del cliente. Un retraso en estas tareas posterga la fecha de entrega final.

Criticidad Media (Importante / No Bloqueante): Incluye tareas necesarias para cumplir con la propuesta de valor y los estándares de calidad del sistema, pero que no detienen el desarrollo ni la operación inmediata.

Criticidad Baja (Deseable / Postergable): Agrupa aquellas actividades, mejoras estéticas o funciones complementarias que aportan valor a largo plazo pero no son obligatorias para el lanzamiento del sistema.

Ficha cronológica:

Requerimientos	Criticidad	Fechas
IT. 1	Alta	Semana 2 y 3
IT. 2	Media	Semana 4 y 5
IT. 3	Alta	Semana 5, 6 y 7
IT. 4	Baja	Semana 8 y 9
IT. 5	Media	Semana 10 y 11